

PA 1: Implementing xv6 User Commands and System Calls

Repository: <https://github.com/gael68/xv6-riscv>, branch `pa1-syscalls`. Based on upstream `mit-pdos/xv6-riscv` (commit `9e3161a`). Built with `riscv64-elf-gcc` and run on `qemu-system-riscv64 11.1.1` (macOS, Apple Silicon). As the assignment requires, `CPUS := 1` is set in the Makefile. One xv6 tick is one timer interrupt, about 0.1 s (`kernel/trap.c` schedules the next timer 1,000,000 cycles ahead).

Task 1. (5 pts) Implement the uptime utility

File: `user/uptime.c` (new); `_uptime` added to `UPROGS` in the Makefile.

The program calls the existing `uptime()` system call and prints its result. `uptime()` is declared in `user/user.h`, has a stub generated in `user/usys.S` from `user/usys.pl`, and is implemented by `sys_uptime()` in `kernel/sysproc.c`, which reads the global `ticks` counter while holding `tickslock`. `main` ends with `exit(0)` because xv6 user programs have no C runtime to return to.

```
int
main(int argc, char *argv[])
{
    printf("up %d clock ticks\n", uptime());
    exit(0);
}
```

Results. Running it twice shows the tick count increasing (about 3 s passed between the two commands):

```
init: starting sh
$ uptime
up 20 clock ticks
$ uptime extra args
up 50 clock ticks
```

Extra arguments are ignored, the same as the Linux `uptime` command.

Task 2. (10 pts) Implement a `time1` command that reports elapsed time

File: `user/time1.c` (new); `_time1`, `_matmul` and `_sleep` added to `UPROGS`. The provided `matmul.c` and `sleep.c` were copied into `user/`.

How it works:

- **Argument handling:** if no command is given (`argc < 2`), print a usage message to `stderr` and exit with status 1.
- **Building `exec()` arguments:** the shell passes a NULL-terminated `argv`, so `argv + 1` is already a valid argument vector for the timed command (`argv[1]` is its name). No copying is needed.

- **Timing:** read `uptime()` before `fork()`. The child calls `exec(argv[1], argv + 1)`. The parent calls `wait()`, then reads `uptime()` again and prints the difference.
- **Validation:** `fork()` failure is reported. If `exec()` returns, it failed; the child prints an error and exits with status 1. `wait()` failure is also checked. The parent collects the child's exit status and prints `command exited with non-zero status N` when it is not 0. This way a failed command is not reported as a normal timing.

Results.

```
$ timel
usage: timel command [args ...]
$ timel nosuchcmd
timel: exec nosuchcmd failed
command exited with non-zero status 1
elapsed time: 0 ticks
$ timel matmul
Time: 6 ticks
elapsed time: 6 ticks
$ timel matmul & timel matmul &
$ Time: 11 ticks
elapsed time: 12 ticks
Time: 10 ticks
elapsed time: 11 ticks
timel matmul 10 & timel matmul 10 &
$ Time: 117 ticks
Time: 115 ticks
elapsed time: 116 ticks
elapsed time: 117 ticks
timel sleep 10
elapsed time: 11 ticks
```

Notes: on this machine one `matmul` run takes about 6 ticks, versus ~28 ticks in the handout, so I also ran `matmul 10` (10 iterations) to get larger, easier-to-read numbers. With two `matmul` jobs in the background, each one takes about twice as long as it does alone, because they share the single CPU. A command typed while background output is printing is echoed after that output, which is why some command lines above appear without a `$` prompt.

Task 3. (10 pts) Keep track of how much cputime a process has used

File	Change	Purpose
kernel/proc.h	Added <code>uint cputime;</code> to <code>struct proc</code> .	Per-process count of timer ticks during which the process was running.
kernel/proc.c (<code>allocproc</code>)	Added <code>p->cputime = 0;</code> next to the <code>pid/state</code> initialization.	Proc slots are reused, so a new process must start at 0 instead of inheriting the old value from <code>proc[]</code> .
kernel/trap.c (<code>usertrap</code>)	In the <code>which_dev == 2</code> (timer) branch, <code>p->cputime++</code> before <code>yield()</code> .	The timer interrupted the process in user mode: it used up its time slice, so charge it one tick.
kernel/trap.c (<code>kerneltrap</code>)	In the timer branch (<code>which_dev == 2 && myproc() != 0</code>),	The timer interrupted the process while it was in the kernel (e.g.

File	Change	Purpose
	<code>myproc()->cputime++</code> before <code>yield()</code> .	inside a system call). That time is on its behalf too.

This is sampling-based accounting. Whichever process is running when the timer fires is charged the whole tick. When the CPU is idle (the scheduler is running and `myproc()` is 0), no process is charged. That is why a sleeping process uses 0 CPU ticks. Only the process itself changes `cputime`, and a parent reads it only after the child is a ZOMBIE (holding `pp->lock`), so the increment needs no extra lock.

Task 4. (10 pts) Implement a `wait2()` system call that waits for a child to exit and returns the child's status and rusage

File	Change	Purpose
kernel/pstat.h (new)	<code>struct rusage { uint cputime; };</code>	Resource-usage record shared by the kernel and user programs.
user/user.h	Added <code>struct rusage;</code> and <code>int wait2(int*, struct rusage*);</code>	Forward declaration and prototype so user programs can call <code>wait2</code> .
user/usys.pl	Added <code>entry("wait2");</code>	Generates the user-space stub in <code>usys.S</code> : loads <code>SYS_wait2</code> into <code>a7</code> and executes <code>ecall</code> .
kernel/syscall.h	<code>#define SYS_wait2 23</code>	New system call number (after <code>SYS_sync</code> , 22).
kernel/syscall.c	<code>extern uint64 sys_wait2(void);</code> and <code>[SYS_wait2] = sys_wait2</code> in <code>syscalls[]</code> .	Dispatches system call 23 to its handler.
kernel/sysproc.c	New <code>sys_wait2():argaddr()</code> fetches both user pointers, then calls <code>kwait2(p, ru)</code> .	Kernel entry point; unpacks arguments from the trapframe.
kernel/proc.c	New <code>kwait2(uint64 addr, uint64 ru)</code> , copied from <code>kwait()</code> , plus <code>#include "pstat.h"</code> .	Waits for a ZOMBIE child and copies its exit status and <code>struct rusage</code> to user memory.
kernel/defs.h	<code>int kwait2(uint64, uint64);</code>	Makes <code>kwait2</code> visible to <code>sysproc.c</code> .

`kwait2()` has the same locking and sleep/wakeup logic as `kwait()`. When it finds a ZOMBIE child, it fills a local `struct rusage` from `pp->cputime` while still holding `pp->lock`. It copies the status to `addr` and the `rusage` to `ru` with `copyout()` (either pointer may be 0 to skip it), and only then frees the child with `freeproc()`. If either `copyout()` fails (bad user pointer), it

releases both locks and returns `-1` without freeing the child, the same as `kwait()`. It also returns `-1` when the caller has no children or has been killed. Note that this version of xv6 renamed `wait/sleep` in the kernel to `kwait/sleep_prepare()+sleep()`, so `kwait2` follows the new API.

```
pid = pp->pid;
usage.cputime = pp->cputime;
if ((addr != 0 &&
    copyout(p->pagetable, p->sz, addr, (char *)&pp->xstate,
            sizeof(pp->xstate)) < 0) ||
    (ru != 0 &&
    copyout(p->pagetable, p->sz, ru, (char *)&usage,
            sizeof(usage)) < 0)) {
    release(&pp->lock);
    release(&wait_lock);
    return -1;
}
```

Task 5. (10 pts) Implement a time command that outputs elapsed time, CPU time, and %CPU used

File: `user/time.c` (new); `_time` added to `UPROGS`.

`time` works like `time1` (same argument handling, `fork/exec`, and error checks), but it calls `wait2(&status, &ru)` instead of `wait()`. It computes `%CPU = cputime × 100 / elapsed` with integer arithmetic, and uses 0% when the elapsed time is 0 ticks to avoid dividing by zero. It includes `kernel/pstat.h` to get the full `struct rusage` definition. As with `time1`, a non-zero exit status from the timed command is reported.

Results.

```
$ time
usage: time command [args ...]
$ time nosuchcmd
time: exec nosuchcmd failed
command exited with non-zero status 1
elapsed time: 0 ticks, cpu time: 0 ticks, 0% CPU
$ time sleep
usage: sleep <ticks>
command exited with non-zero status 1
elapsed time: 0 ticks, cpu time: 0 ticks, 0% CPU
$ matmul
Time: 6 ticks
$ time matmul
Time: 5 ticks
elapsed time: 6 ticks, cpu time: 5 ticks, 83% CPU
$ time matmul & time matmul &
$ Time: 11 ticks
elapsed time: 12 ticks, cpu time: 6 ticks, 50% CPU
Time: 10 ticks
elapsed time: 11 ticks, cpu time: 6 ticks, 54% CPU
time matmul 10
Time: 61 ticks
elapsed time: 61 ticks, cpu time: 61 ticks, 100% CPU
$ time matmul 10 & time matmul 10 &
$ Time: 125 ticks
Time: 123 ticks
elapsed time: 124 ticks, cpu time: 62 ticks, 50% CPU
elapsed time: 125 ticks, cpu time: 63 ticks, 50% CPU
time matmul 10 & time matmul 10 & time matmul 10 &
$ Time: 171 ticks
```

```

elapsed time: 173 ticks, cpu time: 58 ticks, 33% CPU
Time: 171 ticks
Time: 168 ticks
elapsed time: 170 ticks, cpu time: 57 ticks, 33% CPU
elapsed time: 171 ticks, cpu time: 58 ticks, 33% CPU
time sleep 10
elapsed time: 10 ticks, cpu time: 0 ticks, 0% CPU
$ time sleep 100
time echo hello world
elapsed time: 100 ticks, cpu time: 0 ticks, 0% CPU
$ hello world
elapsed time: 0 ticks, cpu time: 0 ticks, 0% CPU

```

Summary of results

Test	Elapsed (ticks)	CPU (ticks)	%CPU	Expected
time matmul 10 (alone)	61	61	100%	CPU-bound, has the CPU to itself → ~100%
2 × time matmul 10 (concurrent)	124 / 125	62 / 63	50% / 50%	Round-robin between 2 → ~50% each
3 × time matmul 10 (concurrent)	173 / 170 / 171	58 / 57 / 58	33% each	Round-robin between 3 → ~33% each
time sleep 10	10	0	0%	Blocked the whole time → 0%
time sleep 100	100	0	0%	Elapsed matches the requested ticks
time echo hello world	0	0	0%	Finishes within one tick
time nosuchcmd	0	0	0%	exec fails; non-zero status reported

The results match the expected behavior. CPU time for one `matmul 10` stays about 57–63 ticks whether it runs alone or with other jobs. Elapsed time grows in proportion to the number of competing jobs, so %CPU is about 1/N. Single-`matmul` runs are short (5–6 ticks), so ±1 tick of sampling error shows up as 83% or 54% instead of exactly 100% or 50%.

Regression check. After all kernel changes, `usertests -q` reports **ALL TESTS PASSED** (61 tests). The `usertrap(): unexpected scause` messages it prints come from the `kernmem` test, which deliberately touches kernel memory and expects to be killed.

Task 6. (5 pts) Summary of what I learned and difficulties

What I learned. I learned the full path of an xv6 system call: the user prototype in `user.h`, the `usys.pl`-generated stub that puts the call number in `a7` and runs `ecall`, `usertrap()` → `syscall()` dispatching through `syscalls[]`, and the handler reading its arguments from the trapframe with `argaddr()/argint()`. I also learned that the kernel cannot dereference user pointers directly and must use `copyout()`. I saw how `fork`, `exec` and `wait` work together to

run and time a command, and how the timer interrupt drives both the `ticks` clock and preemption. That made clear why charging a tick on each timer-driven `yield()` measures CPU time by sampling, and why a sleeping process shows 0% CPU.

Difficulties and how I handled them.

- **The current xv6 differs from the handout's hints.** Upstream xv6-riscv has renamed kernel functions: `wait` → `kwait`, `exit` → `kexit`, the user `sleep` system call → `pause`, and the kernel's `sleep(chan, lock)` → `sleep_prepare()` + `sleep()`. I read the current `kwait()` and `sys_wait()` and wrote `kwait2/sys_wait2` against the new API instead of following older tutorials.
- **Choosing where to count CPU time.** Timer interrupts arrive in both `usertrap()` and `kerneltrap()`. To also count time a process spends in system calls, I increment in both. In `kerneltrap()` I only increment when `myproc() != 0`, so idle time in the scheduler is not charged to any process.
- **`matmul` ran much faster than in the handout** (about 6 ticks instead of about 30), so single runs gave noisy percentages. I used `matmul 10` to get longer runs and clearer percentages.
- **Validation.** My first version of `time` printed normal statistics even when `exec` failed. I changed `time/time1` to check the exit status (which `wait2` returns anyway) and report failures. I also tested the missing-argument, bad-command and zero-elapsed cases.

Extra Credit (5 pts). Limitations of our time command

- **CPU time of the timed command's children is not counted.** `wait2()` returns only the direct child's own `cputime`. If the timed program forks workers (a shell script, `make`, and so on), their CPU time is lost when the timed program reaps them. I demonstrated this by timing `time` itself:

```
$ time time matmul 10
Time: 59 ticks
elapsed time: 59 ticks, cpu time: 59 ticks, 100% CPU      <- inner time (matmul)
elapsed time: 60 ticks, cpu time: 1 ticks, 1% CPU         <- outer time (the inner time
process)
```

The outer `time` sees 1 tick of CPU even though 59 ticks were used on its behalf. A fix is to keep a second counter, e.g. `cputime_children`, in `struct proc`. When `kwait/kwait2` reaps a child, add the child's `cputime` + `cputime_children` to the parent's `cputime_children`, and return the sum in `rusage`. Unix does this with `RUSAGE_CHILDREN/cutime`. Children that are never waited for, or that are reparented to `init`, would still be missed.

- **Coarse resolution (about 100 ms per tick).** Short commands such as `echo` show 0 ticks elapsed and 0 CPU. Every measurement has an error of about ± 1 tick, so short runs can give misleading percentages (e.g. 83% for a CPU-bound `matmul`).
- **Sampling bias in CPU accounting.** A process is charged a whole tick only if it is running at the exact moment of the timer interrupt. A process that always blocks just before the tick is never charged, and a process that wakes just before it is charged the full tick. Time spent handling interrupts (disk, console) is also charged to whichever process was interrupted.

- **%CPU can exceed 100% or be skewed.** Elapsed time is measured by the parent with `uptime()`, which includes `fork/exec` overhead and the parent's own delay before it runs again after the child exits. CPU time is counted separately by the timer. Because the two clocks are sampled at different moments, the ratio can be off by one tick in either direction. Integer division also truncates the result.
- **No user/system split and no other resources.** Unlike Linux `/usr/bin/time`, we report only total CPU ticks: no user-mode vs. kernel-mode time, memory usage, page faults, or I/O counts.
- **Assumes a single CPU.** `ticks` advances only on CPU 0, while `cputime` is incremented on every CPU. With `CPUS > 1`, a multi-process workload could exceed 100% CPU, and the %CPU no longer means "share of the one processor".
- **Output is in ticks, not seconds.** The tick length depends on the timer interval in `trap.c/start.c`, so results can't be compared directly to wall-clock seconds or across machines.

Resources and AI-use disclosure

Resources: the assignment handout; the xv6-riscv source (github.com/mit-pdos/xv6-riscv) and the xv6 book, chapters 2 and 4 (system calls, traps) and chapter 7 (scheduling).

AI use: Claude (Anthropic, via Claude Code) wrote the code for Tasks 1–5 (`uptime.c`, `time1.c`, `time.c`, the `cputime` accounting, and the `wait2` system call). It also built and ran the tests in QEMU and drafted this report from the recorded output. [REVIEW AND EDIT: describe what you checked, changed, and understood yourself, then remove this bracketed note and the red color.]